Lecture 5 · Part I · Lecture notes

Ensembles and random forests

Dataset: Forest CoverType

How to use these notes

The primary text for this lecture is Géron, Chapters 5–6. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 The task, and the member model: a decision tree **2**

1.1 The brief . . . 2

1.2 The data . . . 2

1.3 What a decision tree is . . . 3

1.4 Gini impurity, and the objective CART minimises . . . 3

1.5 Fit one with no constraints . . . 3

1.6 What depth buys, and the tree we can ship . . . 4

2 The result we are carrying forward **4**

2.1 Where many different models come from . . . 4

2.2 When an ensemble of trees is the tool . . . 5

3 The mathematics: the variance of an average of correlated predictors **5**

3.1 Two, then n . . . 5

3.2 Reading the two terms . . . 6

4 Building them, and measuring ρ **6**

4.1 Voting, the simplest ensemble . . . 6
4.2 Bagging and random forests . . . 7
4.3 A validation set you get for nothing . . . 7
4.4 Measuring ρ needs something we do not have . . . 7

5 And what did that do to the accuracy? 8

6 The bill 8

6.1 What the forest changed for the agency . . . 9
6.2 Feature importance, and a control . . . 9

7 Boosting, and what practitioners reach for 9

7.1 A forest in practice . . . 10

8 Did the averaging fix what we diagnosed? 11

9 Design exercise: the probability of a click on an Instagram ad 11

10 The notebook 13

11 Exercises 13

Summary 13

1 The task, and the member model: a decision tree

1.1 The brief

An agency managing a national forest must publish a map of forest cover type — which of seven species dominates each 30 by 30 metre patch — over an area far too large to survey on foot, predicted from cartographic measurements alone (elevation, slope, aspect, distances to water, road and fire ignition points, hillshade, soil type). The map goes into the public record, decides where logging is permitted and which parcels qualify for habitat protection, and a landowner refused a permit is entitled to ask why. So every *individual* prediction must come with a justification a person can read: a statement, in terms of the measured columns, of why this patch was classified as that species, and no longer than eight conditions. Eight is the agency's number. It is a hyperparameter fixed by the brief rather than by cross-validation, and that is worth noticing.

1.2 The data

581,012 patches, each with 54 measurements: 10 quantitative, 4 already one-hot for wilderness area, 40 already one-hot for soil type. No missing values, no text columns, nothing to impute. We use 60,000 patches, stratified so the class proportions are preserved exactly, split 48,000 / 12,000 — stratifying on the label matters when one class is 0.5% of the data. The subsample is a compromise for speed, and every number in this lecture is measured on it. Two species are 85% of the ground, so a classifier that always says "Lodgepole Pine" scores **48.8%**: that is the floor, and the distance above it is the only part you earned.

1.3 What a decision tree is

Every model so far predicts with a weighted sum $\boldsymbol{\theta}^\top \mathbf{x}$. A **decision tree** is not a weighted sum at all: it is a sequence of yes/no questions, each internal node asking $x_k \le t_k$ about one feature, each leaf predicting the majority class of the training instances that reached it. Predicting one instance is one root-to-leaf path, and *the path is the justification*: every condition on it is a measurement a surveyor took. Trees need no feature scaling, because $x_k \le t_k$ is invariant under any strictly increasing transformation of x_k.

1.4 Gini impurity, and the objective CART minimises

A node holds m instances over K classes, with p_k the fraction in class k; the class distribution $\mathbf{p}$ is the only thing an impurity measure sees. **Gini impurity** is

$$G(\mathbf{p}) = 1 - \sum_k p_k^2.$$

Draw two of the node's instances independently with replacement: $\sum_k p_k^2$ is the probability they are the same class, so G is the probability they *differ*. A pure node, with some $p_k = 1$, never disagrees with itself and has $G = 0$.

A question $x_k \le t_k$ sends m_L of the node's m instances left and m_R right. CART scores it by the impurity I of the two children, weighted by their size,

$$J(k, t_k) = \frac{m_\mathrm{L}}{m} I_\mathrm{L} + \frac{m_\mathrm{R}}{m} I_\mathrm{R},$$

and at every node chooses the feature k and threshold t_k that minimise it, searching every feature and every threshold (each threshold midway between two adjacent observed values). Equivalently it maximises the impurity decrease $\Delta I = I_\mathrm{parent} - J$. Entropy, $H(\mathbf{p}) = -\sum_k p_k \log_2 p_k$, is the alternative criterion Scikit-Learn offers; it ranks candidate splits almost identically, so we leave the default.

1.5 Fit one with no constraints

Depth 33, 5,699 leaves, 100.0% training accuracy. With 48,000 training patches that is eight patches per leaf on average, so of course it is perfect on its own data. Cross-validated on the training set it scores 82.1%, and on the test set, once, 82.6%: real, and far above the 48.8%

baseline, so the tree family works here. But count what a prediction rests on. The tree is 33 levels deep and its mean justification is **17.88** conditions; the brief allows eight.

Two hyperparameters take that depth away. `max_depth` is a hard ceiling on the length of every path — and therefore on the justification — and `min_samples_leaf` rejects any split that would leave a child with fewer training instances than its value.

1.6 What depth buys, and the tree we can ship

Cross-validated accuracy is still climbing at depth 18, where it stands **8.0 points** above depth 8. That is the measured price of the eight-condition requirement. Cross-validation does not get a vote, because it is not optimising the thing the agency is buying; the 8.0-point figure goes to the agency, who may trade a condition or two for it.

The readable tree is capped at depth 8 with a minimum leaf of 20, so that each justification rests on at least twenty surveyed patches rather than on one. It has 163 leaves and a mean of **7.97** conditions per prediction, against the unconstrained tree's 17.88, and scores **73.0%** on the test set.

2 The result we are carrying forward

That depth-8 tree is the member model of everything that follows, and this lecture starts with a measurement rather than a score. Refit the same tree, with the same hyperparameters and the same seed, on twenty different 90% subsamples of the same training rows: test accuracy moves by a standard deviation of 0.25 points, and *9.1% of the individual predictions change*. Only 71.4% of test patches get the same answer from all twenty.

A stable metric, an unstable model. This is *variance*: sensitivity of the fitted function to the particular training sample. The whole of today follows from one question: what does averaging do to variance?

Forest CoverType	Test accuracy	Conditions per justification
Always "Lodgepole Pine"	48.8%	0
Depth-8 tree, minimum leaf 20	73.0%	7.97
Unconstrained tree	82.6%	17.88

2.1 Where many different models come from

Ask a difficult question of a thousand random people and aggregate their answers, and the aggregate is often better than an expert's. The same is true of models, and it has a name: an **ensemble**. A group of weak learners can form a strong learner — provided there are enough of them and they are sufficiently *diverse*. "Sufficiently diverse" is doing all the work in that sentence; it is the informal name of a quantity the next section defines.

Why the members are trees: averaging attacks variance, and trees are the high-variance model we have just measured; a tree is cheap to fit, so two hundred are affordable; and an unconstrained tree has *low bias* — which matters, because averaging does nothing whatever to bias, so the members

had better not have any to spare. That third point is why the ensembles here use unconstrained members. It looks reckless after an opening section spent capping the depth at 8, and by the end of the next section it will look obvious.

2.2 When an ensemble of trees is the tool

The data this lecture's model suits has a shape, and it is worth knowing it before the mathematics, because it is the shape of most of the tables a practitioner meets. A table of numbers and categories mixed, on different scales: a split $x_k \le t_k$ needs no scaling and no linear shape. Interactions that nobody can write down — elevation *and* soil type *and* aspect — because a path through a tree *is* an interaction, found rather than specified. Tens of thousands of rows or more, enough to grow deep members whose errors differ. And a task where accuracy matters more than a one-line reason: the vote is accurate, and the single tree was the readable one. Credit scoring, fraud flags, churn, click prediction, risk scores in medicine and nearly every tabular competition have that shape, and on it a random forest or a gradient-boosted ensemble is the first model a practitioner fits. Not images, text or sequences: a pixel or a token is not a column with a meaning of its own, which is why Parts II–IV of this course exist.

3 The mathematics: the variance of an average of correlated predictors

Forget trees for a moment. You have n predictors of the same quantity, each with variance σ^2, and you average them: $\bar{f} = \frac{1}{n}\sum_t f_t$. What is $\mathrm{Var}(\bar{f})$?

The answer everyone gives first is σ^2/n. That step is valid only when the f_t are *uncorrelated* — the variance of a sum is the sum of the variances *plus twice the sum of the covariances*. Dropping the covariances is an assumption, and it is exactly the assumption that fails for an ensemble of trees grown on the same dataset.

3.1 Two, then n

With $\mathrm{Var}(f_1) = \mathrm{Var}(f_2) = \sigma^2$ and $\mathrm{Corr}(f_1, f_2) = \rho$:

$$\mathrm{Var}\left(\tfrac{f_1+f_2}{2}\right) = \tfrac{1}{4}\left[\sigma^2 + \sigma^2 + 2\rho\sigma^2\right] = \frac{\sigma^2(1+\rho)}{2}.$$

At $\rho = 0$ this is the familiar halving; at $\rho = 1$ it is σ^2, because the two predictors are the same predictor and averaging achieves nothing. The general case has both limits in it.

n identically distributed predictors, common pairwise ρ

Assume each has variance σ^2 and $\mathrm{Cov}(f_s, f_t) = \rho\sigma^2$ for $s \neq t$. Then

$$\mathrm{Var}\Big(\sum_{t=1}^{n} f_t\Big) = \sum_{s=1}^{n}\sum_{t=1}^{n} \mathrm{Cov}(f_s, f_t),$$

a double sum of n^2 terms. Split them by whether $s = t$: n diagonal terms each σ^2, and $n(n-1)$ off-diagonal terms each $\rho\sigma^2$. Divide by n^2:

$$\mathrm{Var}(\bar{f}) = \frac{1}{n^2}\big[n\sigma^2 + n(n-1)\rho\sigma^2\big] = \frac{\sigma^2}{n} + \frac{n-1}{n}\rho\sigma^2,$$

and writing $\frac{n-1}{n} = 1 - \frac{1}{n}$ and gathering the $1/n$ terms,

$$\boxed{\mathrm{Var}(\bar{f}) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{n}}$$

Identically distributed, not independent — that is the whole point. n trees grown on bootstrap samples of one dataset have the same distribution and are certainly not independent: they were handed the same dataset.

3.2 Reading the two terms

The sentence to remember

Averaging destroys the independent component of the variance and leaves the correlated one completely untouched.
And note what it does *not* say. It says nothing about bias: an average of n biased predictors is exactly as biased as one of them.

As n grows the second term vanishes, so the variance falls onto the floor $\rho\sigma^2$ that no number of members can touch.

So the way to improve an ensemble is *not more members*. It is less correlated members. And — the point that gets forgotten — anything which lowers ρ by making individual members worse is a trade, not a free lunch, because σ^2 appears in *both* terms and can rise while ρ falls.

One formula, two algorithms

Bagging randomises the rows. A random forest also randomises the columns. Both exist for one purpose: to lower ρ, because $\rho\sigma^2$ is the floor that n cannot reach below.

4 Building them, and measuring ρ

4.1 Voting, the simplest ensemble

A `VotingClassifier` combines different algorithms fitted to the same data — in the deck's example a tree, a logistic regression and a 5-nearest- neighbour classifier. `voting="hard"` takes the majority

of the predicted labels; `voting="soft"` averages the predicted probabilities, and is usually better, because a confident member outweighs several hesitant ones. Different algorithms make different kinds of errors, which is the informal way of saying ρ is small. The formula applies unchanged.

4.2 Bagging and random forests

Bagging (*b*ootstrap *agg*regat*ing*) draws m rows with replacement, fits a tree, and repeats n times. Each bootstrap sample has the size of the training set but only about 63% of its distinct rows; the ensemble aggregates by majority vote for classification and by the mean for regression. Each member is individually worse than one tree fitted to the whole set, and the ensemble is better — that is the formula, not a paradox. The members are unconstrained trees, because averaging cannot remove bias and is about to remove the variance. **Pasting** is the same without replacement, producing less diverse members, so bagging usually wins — but it is a cheap thing to cross-validate rather than assume.

A **random forest** adds a random subset of features at every node — the classification default `max_features="sqrt"` gives $\lfloor\sqrt{54}\rfloor = 7$ of our 54 columns, drawn afresh at each node, so a tree with 200 nodes draws 200 different subsets.

4.3 A validation set you get for nothing

Each bagged member never sees about 37% of the rows — its **out-of-bag** instances. A given row is missed by one draw with probability $1 - 1/m$, so by all m draws with probability

$$\left(1 - \frac{1}{m}\right)^m \longrightarrow e^{-1} \approx 0.368,$$

hence about 63% distinct rows in and 37% held out. Score each member on its own out-of-bag rows and you have a generalisation estimate with no validation split and no extra fit.

Read it carefully, though. It is an average over members, each scored on a different 37%, so it estimates the generalisation of one member's *procedure*, not of the ensemble you ship. It is available only where members see bootstrap samples: with `bootstrap=False` there are no out-of-bag rows and `oob_score=True` raises. Use it to compare configurations cheaply, then confirm the one you pick with cross-validation or the test set. It does not replace the test set — it replaces the validation set.

A leakage detector you get free

If the out-of-bag score comes out much *better* than the test score, something in the pipeline saw the out-of-bag rows anyway — a scaler fitted outside the ensemble, or a feature built from the whole frame.

4.4 Measuring ρ needs something we do not have

ρ is the correlation between two members over the randomness of the whole procedure, which includes *which training set you were handed*. Two trees grown on one dataset are correlated through that dataset — so a single training set cannot show it. Conditional on the data, the members are independent by construction and ρ would measure as zero.

CoverType is large enough to cut 20 *disjoint* training sets of 20,000 rows each, no row reused, and still keep 12,000 aside as a shared test set, then fit a 20-member ensemble on each. Measured that way, ρ is 0.072 for bagging and 0.043 for the random forest: the columns lowered it, as the formula promised, and the measured variance of an average of n members falls on the curve $\rho\sigma^2 + (1-\rho)\sigma^2/n$ for both, with nothing fitted to make it.

5 And what did that do to the accuracy?

200 members, 48,000 training patches	10 members	200 members
Bagging	87.7%	89.8%
Random forest	85.9%	88.4%
The legible depth-8 tree	—	73.0%
Always "Lodgepole Pine"	—	48.8%

Both ensembles beat the readable tree by fifteen points and the unconstrained tree by six. The last 190 members buy about two: most of the gain arrives by the first few dozen members, steep then flat, exactly as $(1-\rho)\sigma^2/n$ says it should.

Now put the two tables beside each other:

Ensemble	ρ	One member's accuracy	Ensemble accuracy
Bagging	0.072	78.8%	89.8%
Random forest	0.043	73.6%	88.4%

The forest lowers ρ — and the ensemble with the higher ρ is the more accurate.

That is not a contradiction — read the formula again

The formula is about *variance*. It says nothing about bias, and accuracy depends on both. Every mechanism that lowers ρ does so by *handicapping the members*. A tree allowed only 7 of 54 columns per node is a worse tree: 73.6% against bagging's 78.8%. And σ^2 appears in both terms, so lowering ρ while raising σ^2 can leave you worse off.

6 The bill

Failure condition — an ensemble has no per-prediction justification

The 200 trees of the forest hold **1,497,919 leaves** between them. The justification for one prediction is now 200 decision paths and a vote. Drawn to scale, against leaves on a logarithmic axis: the legible depth-8 tree scores 73.0%, the unconstrained tree of the opening section 82.6%, and the forest at 1.5 million leaves 88.4% — fifteen points of accuracy, and four orders of magnitude more model to read.

On the vote share: a single tree's `predict_proba` returns the class proportions of one leaf, piecewise constant and often built on a handful of rows. An ensemble averages 200 such vectors from 200

different leaves, so the result varies smoothly and is better calibrated — one thing the agency asked for that averaging improves rather than destroys. But better calibrated is not calibrated: a forest's vote share is systematically pulled towards the middle, because unanimity is rare. If a downstream decision uses the number as a probability, check it against observed frequencies before believing it.

6.1 What the forest changed for the agency

Put the readable tree and the bagged forest side by side on the three things the agency cares about. Test accuracy: 73.0% against 89.8%, about seventeen more patches in every hundred mapped right. Stability: 9.1% of the tree's predictions moved when the survey was resampled, and the forest cut that variance by nine tenths, so the map stops changing when the survey does. What backs one prediction: eight conditions a person can read against 200 paths and a vote. That is the case for the ensemble, and it is why this is the model a practitioner fits on a table. The cost is the justification — and a map does not need one per patch, while a refused permit does. Ship the forest for the map, and the readable tree wherever a person has to answer for the decision.

6.2 Feature importance, and a control

`feature_importances_` sums, over every node that split on a feature, the weighted impurity reduction that split achieved — the impurity decrease ΔI of the opening section, accumulated. It runs, it is fast, and the top of the list is entirely sensible: elevation first, then the distances. Nothing invites suspicion.

Before trusting any ranking

Ask what it would look like on a variable you know carries no information. If you cannot answer, add one and find out.

Add a column of uniform random numbers and refit. It ranks **tenth of fifty-five**.

The repair is to break the column and see what breaks: shuffle one column of the *held-out* set, re-score, and report a spread over repeats (`permutation_importance`). It goes beyond Chapter 6 and is beyond the syllabus, for context; it is here because the alternative is to ship the biased measurement.

On the same model and the same columns, permutation importance on held-out data puts the decoy indistinguishable from zero, where impurity importance put it tenth.

7 Boosting, and what practitioners reach for

Everything so far trains members in parallel and averages them. **Boosting** trains them in *sequence*, each correcting its predecessor. The members are not identically distributed and are certainly not exchangeable, so *today's formula does not apply to boosting*. Bagging reduces variance; boosting reduces bias. Two consequences follow from the sequence: boosting cannot be parallelised across members, and it is much easier to overfit with.

AdaBoost fits a weak learner — typically a tree of depth 1, a decision stump — increases the

weight of the instances it got wrong, fits the next learner on the reweighted data, and repeats; it predicts by a weighted vote, each learner weighted by its own accuracy. If it overfits, reduce `n_estimators` or `learning_rate`, or regularise the base estimator.

Gradient boosting keeps the sequence but changes the correction: instead of reweighting the instances, each new learner is fitted to the *residual errors* of the ensemble so far. `learning_rate` scales each member's contribution; a low rate needs more members and generalises better, which is called **shrinkage**. Too few members underfits and too many overfits, so use early stopping (`n_iter_no_change`): the number of members is not a "more is better" parameter, which is exactly the opposite of bagging.

Failure condition — the intuition that does not carry across

Adding trees to a bagged ensemble can only help, because $(1-\rho)\sigma^2/n$ falls monotonically. Adding trees to a *boosted* ensemble eventually hurts. Carrying the first intuition into the second is one of the commonest mistakes with these models.

`HistGradientBoostingClassifier` bins each feature into at most 255 buckets before growing anything, so a split search costs $O(\text{bins})$ rather than $O(m)$. It is dramatically faster on a dataset this size, it is the algorithm LightGBM and XGBoost implement, it handles missing values and categorical columns natively — no imputer, no encoder, provided a categorical column has at most `max_bins` (by default 255) categories, or it raises an error — and on tabular data it is the usual first choice and on most benchmarks the strongest single model. On this dataset, at a budget that fits a lecture, it lands several points below the bagged forest — usual is not always.

Stacking trains a model to do the aggregation, with the base learners' predictions as the input features of a *blender*. The blender must be trained on predictions the base learners made for instances they did not see; `StackingClassifier` does this with cross-validation internally, which is the only reason it is safe to use. Build it by hand and the blender sees in-sample predictions, learns to trust whichever base learner overfits hardest, and the ensemble comes out worse than its best member — with no error and no warning.

7.1 A forest in practice

The recipe that usually works, before any tuning: `RandomForestClassifier(n_estimators=300, max_features="sqrt", oob_score=True, n_jobs=-1)` and `HistGradientBoostingClassifier(learning_rate=0.05, max_iter=500, early_stopping=True)`. No scaling; `HistGradientBoosting` takes missing values and categories as they are. For a forest, more members never hurt, so set n by the budget and read the out-of-bag score as the validation. For boosting, tune the learning rate and let early stopping pick the number of rounds. Under imbalance, `class_weight="balanced"` and per-class recall beside the accuracy. Then the honest part, unchanged since the first lectures: cross-validate the choice, and read the test set once.

Situation	Reach for
A high-variance base learner and time to spare	bagging or a random forest
Many weakly informative columns	a random forest — feature subsampling earns its keep
You want the strongest tabular model	histogram gradient boosting — usually, and measure it against a forest anyway
Several strong, different models and time to validate a blender	stacking — `StackingClassifier`, whose blender sees only cross-validated predictions
You must justify individual predictions	none of them — go back to one tree

The last row is what this lecture exists to teach, and it is the row people skip.

8 Did the averaging fix what we diagnosed?

The diagnosis was *instability*, not inaccuracy — so test the thing that was diagnosed, not the thing that is easy to measure. On 20 disjoint training sets, a random forest's variance of one member's correctness is 0.16734 and of the average of 20 is 0.01508: 91.0% of the variance removed, against a floor of 0.00721 it cannot reach below. Nine tenths of a single member's variance, gone — and we knew before starting both that it would stop and roughly where. The notebook repeats the twenty-subsample disagreement measurement of the second section on a forest; predict the answer before you run it.

9 Design exercise: the probability of a click on an Instagram ad

The lecture closes with a design exercise in the manner of an ML system-design interview, on a task with the shape of data this model suits.

The task. Every time a feed is loaded, an auction chooses which ad fills the slot. For each candidate (user, ad, context) the system must estimate, in a few milliseconds, the probability of a click. The auction ranks candidates by bid × probability, so the number has to be a probability, not a score: a model that ranks well but misprices every bid loses money on every impression. Given: the log of past impressions — who saw which ad, where, when, and whether a click followed — plus what is known about the user, the ad and the moment. Billions of impressions a day; clicks are rare, of the order of one impression in a hundred. The output feeds the auction, budget pacing and the advertiser's reports.

1. **Frame it honestly.** One row per impression; the label is a click within the attribution window; the output is $p(\text{click} \mid \text{user, ad, context})$. Score with a proper scoring rule — log loss — read beside its anchor, the log loss of always predicting the base rate, with AUC beside it for the ranking and never instead. The baseline is the ad's own historical click rate, shrunk towards its campaign's when it has few impressions. *Pitfall:* with one click in a hundred, accuracy says nothing, and a model can have a splendid AUC while every probability it emits is twice too large.
2. **The label is not as clean as it looks.** A click can arrive minutes or hours after the impression,

so a row logged too early is a negative that is not one: label only impressions older than the attribution window, or model the delay. Bots and accidental taps are clicks in the log and not in the world: filter with the platform's invalid-traffic rules first. An ad logged below the fold was never a candidate for a click: keep viewable impressions only. Write the label's definition down before the first fit; every later number is a number about that label.

3. **Split by time.** Train on the days up to a cut, validate on the next day, test on the day after, with a gap at least as long as the attribution window; choose everything on the validation day and read the test day once. *Pitfall:* a random split puts the same user, campaign and hour on both sides; the offline score climbs and the gain vanishes the day the model goes live.

4. **Features.** User: coarse demographics and past behaviour as counts and rates. Ad: campaign, advertiser, creative type, its own history. Context: position, time of day, device, connection. No scaling and no manual crosses — a path through a boosted tree *is* "this advertiser × this device × this hour". *Pitfall:* an ad's click rate computed over the whole log contains the row's own click and every click after it. Compute every history feature *as of* the impression, from the past only, with the code that will compute it at serving time; a feature the serving system cannot produce in time is not a feature.

5. **One click in a hundred.** Keep every click and a random fraction of the non-clicks, and training fits the budget; the fitted probabilities are then too high by a known amount, so shift the log-odds back by the log of the sampling rate, or refit the calibration on an unsampled slice. *Pitfall:* `class_weight="balanced"` fixes the ranking and wrecks the probabilities — a tool for a classifier that outputs a label, where this system outputs a price.

6. **The model.** Tabular rows, mixed types, interactions nobody can write down, tens of millions of rows after sampling: the case for an ensemble of trees. Boosting over bagging here, because shallow members are cheap to evaluate and the model is evaluated billions of times a day. Tune the learning rate on the validation day and let early stopping choose the number of rounds. *Pitfall:* more rounds always lower the training loss and, past a point, raise the next day's — the opposite of bagging. What the large platforms do next, the trees as feature generators for a linear or neural layer, starts from this model.

7. **Calibration.** Bucket the validation day by predicted probability and compare each bucket's prediction with its observed click rate: the reliability diagram, now a business requirement. If it bends, fit a calibration layer — isotonic or Platt — on a slice the model never trained on, and check it per segment. *Pitfall, the cold start:* a new ad has no history; back it off to its campaign's and advertiser's rates, and give new ads a small, deliberate share of impressions so that their own rate can be learned.

8. **The model chooses its own training data.** The log holds only the impressions the previous model chose to show, so an ad it never shows gets no clicks and the next model never learns to show it. Keep a small randomised slice of impressions outside the model's choice; retrain daily; monitor the input distributions and the calibration against a base rate that drifts. *Pitfall:* an offline gain is a hypothesis. The only test that counts is an A/B test on live traffic, read on revenue and on what users do afterwards, with the budgets of both arms held equal.

9. **Ship it.** Log → viewable impressions, invalid traffic out, labels after the window → time split → history features as of each impression → sampled negatives → boosted trees with early stopping → probabilities corrected and calibrated → log loss and reliability against the

anchor → A/B test → retrain and monitor.

10 The notebook

What to change

Set `max_features=None` on the random forest, which turns it back into bagging. Watch ρ rise, each member get better, and the ensemble accuracy rise with it — the trade of the whole lecture, in one keyword.

11 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 6's deck and in the exercise book.

1. For n predictors each of variance σ^2 and pairwise correlation ρ, the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$. State the limit as $n \to \infty$, and what it means for ensembles. [5]
2. Bagging measures $\rho = 0.072$ with one member at 78.8% accuracy; the random forest measures $\rho = 0.043$ with one member at 73.6%. Yet bagging's ensemble is the more accurate. Explain, using the formula above. [5]
3. You set `bootstrap=False` and `oob_score=True`. State what happens and why. [3]
4. Impurity importance ranks a column of random numbers tenth of 55; permutation importance on held-out data puts it at zero. Describe how permutation importance is computed, and why a random column cannot score well on it. [5]
5. A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a contradiction? Answer with a reason. [4]

Summary

A decision tree hands the agency an individual justification in the measured columns, because its root-to-leaf path *is* the justification; CART grows one by choosing at every node the split that minimises the size-weighted Gini impurity of the two children. Unconstrained, the tree scores 82.6% on the test set with 17.88 conditions per prediction; capped at depth 8 for the brief's eight, it scores 73.0% with 7.97. That depth-8 tree refitted on twenty 90% subsamples moved its accuracy by 0.25 points while 9.1% of its predictions changed: variance. From the double sum of covariances, $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/n$: averaging destroys the independent component of the variance and cannot touch the correlated one, so the way to improve an ensemble is less correlated members, not more of them.

This is the model for a table — no scaling, interactions found rather than specified, strong defaults — and the first a practitioner fits on credit, fraud, churn or click data. Bagging randomises the

rows, a random forest also the columns. Measured on 20 disjoint training sets, ρ fell from 0.072 for bagging to 0.043 for the forest, with the measured variances on the curves the formula predicts. The lowest-ρ ensemble was not the most accurate, because every mechanism that lowers ρ also weakens the members and σ^2 sits in both terms. About fifteen points of accuracy cost 1,497,919 leaves and the per-prediction justification the agency's brief was built around. Impurity importance ranked a column of pure noise 10th of 55; a permutation measurement on held-out data put it at zero. Boosting trains its members in sequence and reduces bias, so the formula does not apply to it. The closing design exercise built the click-probability model behind an ad auction in nine steps — a proper score against its anchor, a label defined after the attribution window, a split by time, as-of features, sampled negatives with the log-odds put back, boosted trees stopped early, calibration, a randomised slice and an A/B test — each step named for the pitfall it avoids.